

DevPulse

Архитектурная карта микросервисной соцсети для программистов

FastAPI · gRPC · KrakenD · Kafka · RabbitMQ · Celery · Redis · PostgreSQL · Prometheus · Grafana

Назначение документа

Это техническая карта проекта: какие сервисы делать, зачем каждый нужен, какие технологии где применять, как строить очередность разработки и какие темы закрывать для роста до Middle-track. Документ намеренно подробный: он нужен как маршрут, чтобы не уходить в хаотичное расширение проекта.

Главное ограничение

Проект не должен превращаться в микросервисы ради микросервисов. Каждая технология должна иметь понятную роль: Kafka - поток доменных событий, RabbitMQ/Celery - фоновые задачи, Redis - кэш и временное состояние, PostgreSQL - основное хранилище каждого сервиса.

Оглавление

1. 1. Цель проекта и продуктовая идея
2. 2. Общая архитектура
3. 3. Роли технологий: Kafka, RabbitMQ, Celery, Redis, gRPC, KrakenD
4. 4. Карта микросервисов
5. 5. Подробные сервисы: стек, ответственность, БД, события
6. 6. Межсервисное взаимодействие и gRPC
7. 7. Clean Architecture внутри сервиса
8. 8. Ключевые сценарии работы системы
9. 9. PostgreSQL как отдельный учебный трек
10. 10. Observability: Prometheus, Grafana, Alertmanager
11. 11. High-load часть и нагрузочные тесты
12. 12. Репозиторий и структура проекта
13. 13. Очередность разработки
14. 14. MVP, полная версия и то, что не тащить в первую итерацию
15. 15. Чеклисты, риски и критерии готовности

1. Цель проекта и продуктовая идея

DevPulse - учебно-портфолио проект: микросервисная социальная сеть для программистов. Его задача - не заменить реальную соцсеть, а показать инженерное мышление уровня Middle-track: границы сервисов, событийную архитектуру, PostgreSQL, кеширование, асинхронность, observability и нагрузочное мышление.

Что умеет пользователь

- зарегистрироваться и войти в аккаунт;
- заполнить профиль разработчика: стек, bio, ссылки, уровень, город;
- подписываться на других разработчиков;
- публиковать посты и code snippets;
- добавлять теги к постам;
- ставить реакции и сохранять посты в bookmarks;
- комментировать посты и получать уведомления;
- читать персональную ленту;
- смотреть hot posts и активные теги;
- искать посты по тегам и тексту.

Техническая цель

- построить несколько FastAPI-сервисов с чистыми границами;
- использовать KrakenD как внешний API Gateway;
- показать gRPC для внутренних batch-вызовов;
- использовать Kafka для доменных событий;
- использовать RabbitMQ + Celery для фоновых задач;
- использовать Redis как cache/rate-limit/idempotency storage;
- глубоко поработать с PostgreSQL: индексы, транзакции, EXPLAIN ANALYZE, cursor pagination, outbox;
- добавить Prometheus, Grafana и Alertmanager;
- провести нагрузочные тесты и описать оптимизации в README.

Главный принцип

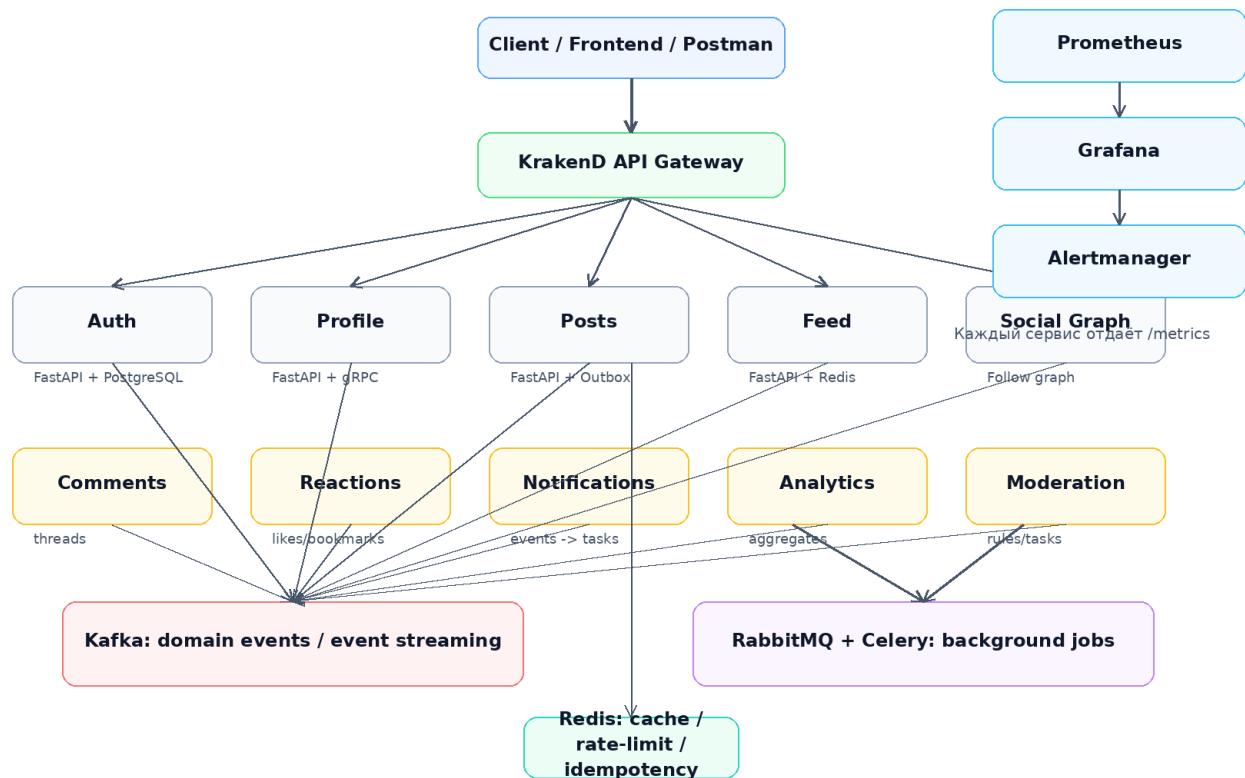
Сначала сделать маленькую, но завершённую production-like систему. Потом расширять её по слоям. Не начинать Kubernetes, ClickHouse, Elasticsearch и frontend до того, как живой backend-контур уже работает.

2. Общая архитектура

Снаружи система выглядит как один API через gateway. Внутри это набор независимых сервисов, каждый со своей базой данных и своей зоной ответственности. Сервисы не должны напрямую лезть в чужие таблицы.

DevPulse: общая архитектура

Внешний HTTP проходит через KrakenD. Межсервисные события идут через Kafka. Фоновые задачи - RabbitMQ/Celery.



Ключевые потоки

- Внешний HTTP:** Client -> KrakenD -> нужный FastAPI-сервис.
- Доменные события:** сервис публикует событие в Kafka, остальные реагируют независимо.
- Фоновые задачи:** сервис ставит задачу в RabbitMQ, Celery worker выполняет её с retry/ack.
- Кэш:** Redis используется для ленты, профилей, rate-limit, idempotency keys и hot data.
- Метрики:** каждый сервис отдаёт /metrics, Prometheus собирает, Grafana визуализирует, Alertmanager уведомляет.

3. Роли технологий

Технология	Роль в проекте	Где использовать	Где не использовать
Kafka	Поток доменных событий	PostCreated, UserFollowed, CommentCreated, ReactionAdded	Не для одиночной отправки email как task queue
RabbitMQ	Рабочая очередь задач	send_email, generate_export, moderate_post	Не как event log всей системы
Celery	Python worker layer поверх брокера	retry, routing, scheduled/background jobs	Не как замена Kafka для доменных событий
Redis	Быстрое временное состояние	cache, rate limit, idempotency, hot feeds	Не как основная БД для критичных данных
gRPC	Быстрые внутренние контракты	GetProfilesBatch, GetFollowees, ValidateToken	Не для каждого мелкого вызова внутри ленты
KrakenD	Внешний API Gateway	routing, rate-limit, aggregation, JWT check	Не для бизнес-логики
PostgreSQL	Надёжное основное хранилище	свои таблицы у каждого сервиса	Не одна общая БД на все сервисы

Kafka

Kafka в DevPulse отвечает за поток доменных событий. Событие означает: “в системе уже что-то произошло”. Например: пользователь зарегистрировался, пост создан, реакция добавлена, комментарий опубликован.

Domain events examples:

- UserRegistered
- ProfileUpdated
- PostCreated
- PostDeleted
- CommentCreated
- ReactionAdded
- UserFollowed
- UserUnfollowed

RabbitMQ + Celery

RabbitMQ и Celery нужны для фоновых задач. Задача означает: “сделай конкретную работу”. Например: отправить письмо, пересчитать рейтинг, сгенерировать экспорт, проверить пост правилами модерации.

Background jobs examples:

- send_email_notification
- send_telegram_notification
- moderate_post
- recalculate_user_score
- generate_profile_export

Redis

Redis не должен быть “ещё одной базой”. В проекте он нужен как быстрый слой для временных данных и read-optimization.

- feed cache;
- profile cache;
- rate-limit counters;
- idempotency keys;
- hot posts cache;
- online status;
- short-lived token/session blacklist.

4. Карта микросервисов

Сервис	Основная роль	API	Хранилище	Очереди/события
Gateway	Внешний вход в систему	HTTP через KrakenD	-	-
Auth	Пользователи, токены, роли	HTTP	PostgreSQL, Redis	Kafka producer
Profile	Профили разработчиков	HTTP + gRPC	PostgreSQL, Redis	Kafka consumer/producer
Post	Посты, сниппеты, теги	HTTP	PostgreSQL, Redis	Kafka producer
Social Graph	Подписки, блокировки	HTTP + gRPC	PostgreSQL, Redis	Kafka producer
Feed	Персональная лента	HTTP	PostgreSQL, Redis	Kafka consumer
Comment	Комментарии	HTTP	PostgreSQL	Kafka producer
Reaction	Лайки, реакции, bookmarks	HTTP	PostgreSQL, Redis	Kafka producer
Notification	Уведомления	HTTP/internal	PostgreSQL, Redis	Kafka consumer + RabbitMQ/Celery
Analytics	Статистика и агрегаты	HTTP/internal	PostgreSQL, Redis	Kafka consumer
Moderation	Проверка контента	HTTP/internal	PostgreSQL	Kafka consumer + Celery

5. Подробные сервисы

5.1 API Gateway Service

Стек: KrakenD

Что делает:

- принимает внешние HTTP-запросы
- проверяет JWT на gateway уровне
- проксирует запросы во внутренние сервисы
- делает rate limiting
- может агрегировать ответы
- скрывает внутреннюю топологию сервисов

Внешние endpoint'ы:

```
POST /api/v1/auth/register
POST /api/v1/auth/login
GET /api/v1/me
GET /api/v1/users/{id}
POST /api/v1/posts
GET /api/v1/feed
POST /api/v1/users/{id}/follow
GET /api/v1/notifications
```

Нюанс

Gateway должен быть тонким. Не переносить туда бизнес-логику, проверки доменных правил и работу с БД.

5.2 Auth Service

Стек: FastAPI, PostgreSQL, SQLAlchemy 2 async, Alembic, Redis, JWT, Argon2/bcrypt, Kafka producer, Prometheus metrics

Что делает:

- регистрация
- логин
- refresh/access tokens
- logout
- password hashing
- basic RBAC
- публикация UserRegistered

Основные таблицы:

```
users
refresh_sessions
roles
user_roles
```

События:

```
UserRegistered
UserLoggedIn
UserBlocked
```

Что прокачивает:

- JWT и security dependencies
- уникальные индексы
- транзакции
- Redis blacklist/session invalidation

- FastAPI Depends

5.3 Profile Service

Стек: FastAPI, gRPC server, PostgreSQL, SQLAlchemy 2 async, Redis cache, Kafka consumer/producer

Что делает:

- профиль разработчика
- bio
- стек технологий
- GitHub/Telegram/сайт
- уровень junior/middle/senior
- публичная карточка пользователя

Основные таблицы:

```
profiles
profile_skills
skills
profile_links
```

gRPC методы:

```
GetProfile(user_id)
GetProfilesBatch(user_ids)
```

Что прокачивает:

- gRPC server
- batch-запросы
- кэш профиля
- денормализация preview-данных

5.4 Post Service

Стек: FastAPI, PostgreSQL, SQLAlchemy 2 async, Alembic, Kafka producer, Redis cache, gRPC client к Profile Service

Что делает:

- создание постов
- редактирование
- удаление
- code snippets
- теги
- черновики
- видимость постов
- outbox-события

Основные таблицы:

```
posts
post_tags
tags
post_code_snippets
post_outbox
```

События:

```
PostCreated
PostUpdated
PostDeleted
PostTagged
```

Что прокачивает:

- composite indexes
- partial indexes
- GIN index
- cursor pagination
- транзакция post + outbox
- EXPLAIN ANALYZE

5.5 Social Graph Service

Стек: FastAPI, PostgreSQL, SQLAlchemy 2 async, Kafka producer, Redis cache, gRPC server

Что делает:

- подписаться
- отписаться
- список подписчиков
- список подписок
- проверка IsFollowing
- блокировки пользователей

Основные таблицы:

```
follows
blocks
```

gRPC методы:

```
GetFollowees(user_id)
GetFollowers(user_id)
IsFollowing(source_user_id, target_user_id)
```

События:

```
UserFollowed
UserUnfollowed
UserBlocked
```

Что прокачивает:

- уникальный индекс follower/followee
- idempotent endpoint
- race condition при двойном follow
- кэш followees

5.6 Feed Service

Стек: FastAPI, PostgreSQL, Redis, Kafka consumer, gRPC client, asyncio

Что делает:

- персональная лента
- лента по тегам
- hot posts
- cursor pagination
- fanout-on-write/fanout-on-read
- кэширование ленты

Основные таблицы:

```
feed_items
user_feed_state
hot_posts
```

События:

```
PostCreated
UserFollowed
```



```
UserUnfollowed
ReactionAdded
CommentCreated
```

Что прокачивает:

- Redis cache strategy
- cursor pagination
- denormalized read model
- consumer idempotency
- high-load read optimization

5.7 Comment Service

Стек: FastAPI, PostgreSQL, Kafka producer, Redis cache

Что делает:

- комментарии к постам
- древовидные ответы
- soft delete
- счётчики комментариев
- события для уведомлений

Основные таблицы:

```
comments
comment_reactions
```

События:

```
CommentCreated
CommentDeleted
```

Что прокачивает:

- recursive CTE или materialized path
- индексы
- anti-spam rate limits
- soft delete

5.8 Reaction Service

Стек: FastAPI, PostgreSQL, Redis, Kafka producer

Что делает:

- лайки
- реакции
- bookmarks
- счётчики реакций

Основные таблицы:

```
post_reactions
post_reaction_counters
bookmarks
```

События:

```
ReactionAdded
ReactionRemoved
PostBookmarked
```

Что прокачивает:

- денормализованные счётчики
- unique constraints
- idempotency
- read optimization

5.9 Notification Service

Стек: FastAPI, PostgreSQL, Kafka consumer, RabbitMQ, Celery, Redis

Что делает:

- уведомления о комментариях
- уведомления о подписках
- уведомления о реакциях
- email/telegram/mock push
- unread notifications

Основные таблицы:

```
notifications
notification_settings
```

События:

```
слушает UserFollowed
слушает CommentCreated
слушает ReactionAdded
```

Celery tasks:

```
send_email_notification
send_telegram_notification
mark_digest_ready
```

Что прокачивает:

- Kafka event -> DB notification -> Celery task
- retry
- dead-letter queue
- идемпотентная обработка событий

5.10 Analytics Service

Стек: FastAPI, Kafka consumer, PostgreSQL, Redis, Prometheus custom metrics

Что делает:

- считает просмотры
- считает популярные теги
- считает активность пользователей
- строит агрегаты для админки
- пишет read models

Основные таблицы:

```
post_views
daily_user_activity
daily_tag_stats
```

События:

```
PostCreated
PostViewed
ReactionAdded
CommentCreated
UserRegistered
```

Что прокачивает:

- batch insert
- aggregation queries
- partitioning по дате
- materialized views
- EXPLAIN ANALYZE

5.11 Moderation Service

Стек: FastAPI, Kafka consumer, RabbitMQ, Celery, PostgreSQL

Что делает:

- простая модерация постов
- стоп-слова
- suspicious content score
- ручная проверка
- блокировка спама

Основные таблицы:

```
moderation_cases
blocked_terms
```

События:

```
слушает PostCreated
публикует PostFlagged
```

Что прокачивает:

- rules engine без ML
- фоновые проверки
- очередь задач
- ручные статусы

6. Межсервисное взаимодействие и gRPC

gRPC нужен не для того, чтобы “всё было модно”, а для нескольких плотных внутренних контрактов. Самый полезный пример - batch-запросы, где HTTP по одному объекту приведёт к N+1 между сервисами.

Хорошие gRPC-контракты	Почему
ProfileService.GetProfilesBatch(user_ids)	Feed/Post могут получить preview авторов пачкой
SocialGraphService.GetFollowees(user_id)	Feed может быстро узнать подписки пользователя
SocialGraphService.IsFollowing(source_user_id, target_user_id)	Проверка доступа/отношений без знания чужой БД
AuthService.ValidateToken / IntrospectToken	Внутренняя проверка токена при необходимости

Плохой паттерн	Почему плохо
Feed -> Profile на каждый feed item	N+1 по сети, высокая задержка и хрупкость
Reaction -> Post на каждый лайк	sync-сцепление сервисов
Post -> Feed прямым вызовом после создания	лучше событие PostCreated через Kafka
Gateway как место бизнес-логики	сервисные правила разползутся в конфиг gateway

Правило

Если один сервис не может работать без синхронного ответа другого сервиса в каждом запросе, проверь границы. Возможно, нужен event-driven read model, денормализация или batch-запрос.

7. Clean Architecture внутри сервиса

Внутри каждого сервиса структура должна быть однотипной. Это сильно упростит поддержку, тесты и понимание проекта на собеседовании.

```
services/post_service/
├── app/
│   ├── main.py
│   ├── config.py
│   ├── domain/
│   │   ├── entities.py
│   │   ├── value_objects.py
│   │   └── exceptions.py
│   ├── application/
│   │   ├── use_cases/
│   │   │   ├── create_post.py
│   │   │   ├── update_post.py
│   │   │   └── delete_post.py
│   │   ├── dto.py
│   │   └── ports.py
│   ├── infrastructure/
│   │   ├── db/
│   │   │   ├── models.py
│   │   │   ├── repositories.py
│   │   │   └── session.py
│   │   ├── kafka/
│   │   │   ├── producer.py
│   │   │   └── outbox.py
│   │   └── redis/
│   │       └── cache.py
│   ├── presentation/
│   │   ├── http/
│   │   │   ├── routes.py
│   │   │   └── schemas.py
│   │   └── grpc/
│   │       └── server.py
│   └── tests/
├── alembic/
├── Dockerfile
└── pyproject.toml
```

Направление зависимостей

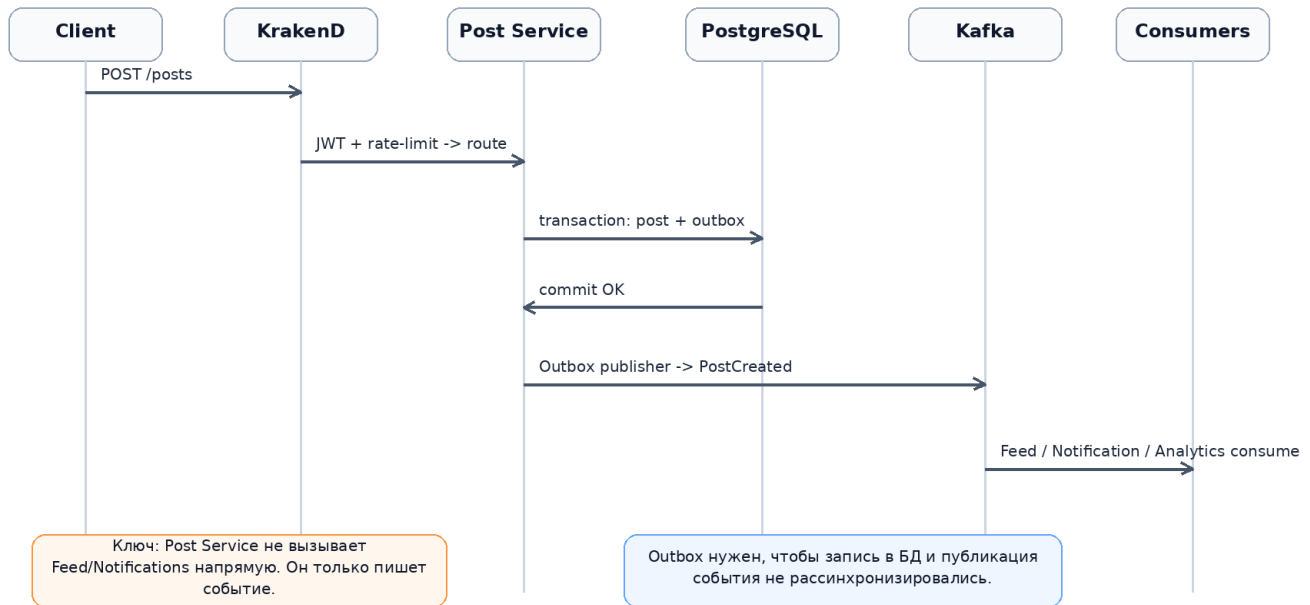
```
presentation -> application -> domain
infrastructure -> application/domain через интерфейсы
domain ни от чего не зависит
```

- **domain:** сущности, value objects, доменные ошибки, чистая логика без FastAPI/SQLAlchemy;
- **application:** use cases, порты репозитория, DTO, orchestration;
- **infrastructure:** SQLAlchemy, Redis, Kafka, RabbitMQ, внешние клиенты;
- **presentation:** HTTP routes, Pydantic schemas, gRPC handlers.

8. Ключевые сценарии работы системы

8.1 Создание поста

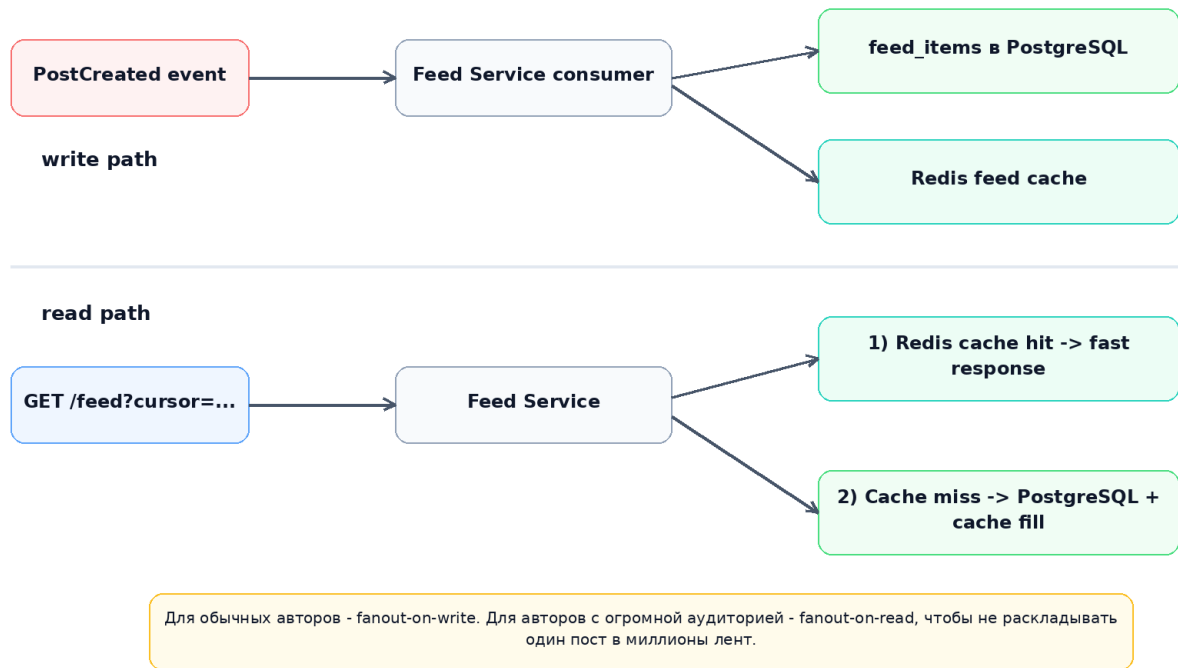
Сценарий: создание поста



1. Client отправляет POST /api/v1/posts в KrakenD.
2. KrakenD проверяет JWT, применяет rate limit и проксирует запрос в Post Service.
3. Post Service валидирует Pydantic schema и вызывает CreatePostUseCase.
4. Use case открывает транзакцию, пишет post в PostgreSQL и событие PostCreated в outbox.
5. После commit outbox publisher публикует PostCreated в Kafka.
6. Feed Service читает событие и добавляет feed_items подписчикам автора.
7. Notification Service читает событие и создаёт уведомления при необходимости.
8. Analytics Service обновляет счётчики активности.

8.2 Чтение ленты

Лента: fanout-on-write + cache



1. Client запрашивает GET /api/v1/feed?cursor=...
2. KrakenD отправляет запрос в Feed Service.
3. Feed Service проверяет Redis.
4. При cache hit ответ отдаётся быстро.
5. При cache miss Feed Service читает feed_items из PostgreSQL по cursor pagination.
6. При необходимости получает профили авторов batch gRPC-вызовом или использует денормализованный snapshot.
7. Ответ кешируется в Redis на короткий TTL.

8.3 Уведомления

```
UserFollowed / CommentCreated / ReactionAdded
    ↓
Kafka topic
    ↓
Notification Service consumer
    ↓
PostgreSQL: notifications
    ↓
RabbitMQ task
    ↓
Celery worker: send_email / send_telegram / mock_push
```

Нюанс уведомлений

Kafka сообщает Notification Service, что событие произошло. RabbitMQ/Celery выполняют конкретную работу. Не смешивать эти уровни.

9. PostgreSQL как отдельный учебный трек

PostgreSQL в этом проекте должен быть не просто “подключённой БД”. Это отдельный трек прокачки: схема, индексы, транзакции, explain, оптимизация запросов, outbox и миграции.

Обязательные темы

- UUID primary keys;
- created_at / updated_at;
- soft delete;
- foreign keys внутри сервиса;
- unique constraints;
- composite indexes;
- partial indexes;
- GIN indexes;
- cursor pagination вместо offset на больших таблицах;
- transactions;
- row-level locks;
- deadlock examples;
- EXPLAIN и EXPLAIN ANALYZE;
- outbox pattern;
- partitioning для views/events по дате.

Примеры индексов

```
CREATE INDEX idx_posts_author_created_at
ON posts (author_id, created_at DESC);

CREATE INDEX idx_feed_user_created_at
ON feed_items (user_id, created_at DESC);

CREATE UNIQUE INDEX uq_follows_pair
ON follows (follower_id, followee_id);

CREATE INDEX idx_posts_published_only
ON posts (created_at DESC)
WHERE status = 'published';
```

Outbox pattern

Outbox нужен, чтобы запись бизнес-сущности и публикация события не расходились. Вместо “сначала записали пост, потом попробовали отправить событие в Kafka” сервис пишет post и outbox_event в одной транзакции. Отдельный publisher позже отправляет событие в Kafka и помечает его published.

```
BEGIN;
  INSERT INTO posts (...);
  INSERT INTO post_outbox (event_type, payload, status)
  VALUES ('PostCreated', '{...}', 'pending');
COMMIT;

-- отдельный publisher:
SELECT * FROM post_outbox
WHERE status = 'pending'
ORDER BY created_at
LIMIT 100
FOR UPDATE SKIP LOCKED;
```

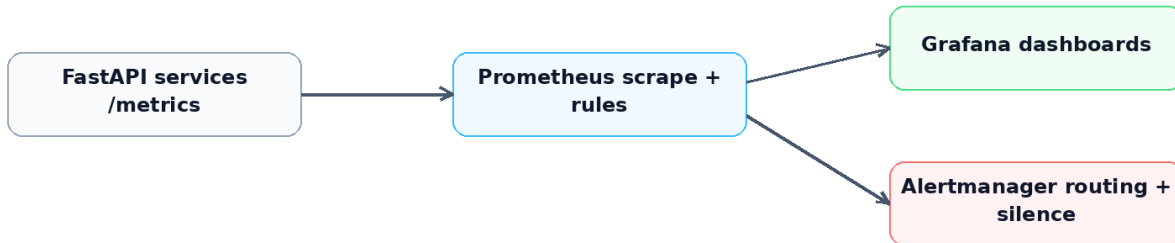
Что показать в README по PostgreSQL

- схему таблиц по каждому сервису;
- почему выбран именно такой индекс;
- до/после EXPLAIN ANALYZE для тяжёлого запроса;
- как работает cursor pagination;

- какие транзакции используются;
- как outbox защищает от рассинхронизации БД и Kafka.

10. Observability

Observability контур



Минимальный набор метрик:

- http_request_duration_seconds
- http_requests_total
- db_query_duration_seconds
- redis_cache_hits_total / misses_total
- kafka_consumer_lag
- celery_queue_length
- outbox_pending_events_total

Метрики каждого FastAPI-сервиса

/metrics

```

http_requests_total
http_request_duration_seconds
db_query_duration_seconds
redis_cache_hits_total
redis_cache_misses_total
kafka_messages_consumed_total
celery_tasks_total
outbox_pending_events_total
  
```

Grafana dashboards

- API Gateway overview;
- Auth Service dashboard;
- Post Service dashboard;
- Feed Service dashboard;
- Kafka lag dashboard;
- RabbitMQ/Celery dashboard;
- PostgreSQL dashboard;
- Redis dashboard.

Alert rules

```

HighErrorRate
P95LatencyTooHigh
PostgresConnectionsHigh
KafkaConsumerLagHigh
CeleryQueueTooLong
OutboxEventsStuck
  
```


Минимум для первой версии

Не нужно сразу делать идеальный observability stack. Достаточно /metrics, Prometheus scrape, один Grafana dashboard и 3-5 простых alert rules. Потом расширять.

11. High-load часть и нагрузочные тесты

High load в портфолио лучше показывать честно: не обещать “миллионы RPS”, а сделать воспроизводимую локальную нагрузку, измерить p95, найти узкое место, оптимизировать и зафиксировать результат.

Что нагружать

```
GET /feed
POST /posts
POST /posts/{id}/reactions
POST /users/{id}/follow
GET /posts?tag=python
```

Инструменты

- k6 или Locust;
- seed script для генерации данных;
- Prometheus metrics;
- Grafana dashboard;
- EXPLAIN ANALYZE отчёты;
- README с методикой замера.

Тестовые данные

Уровень	Пользователи	Посты	Подписки	Реакции
Стартовый	10 000	100 000	500 000	1 000 000
Расширенный	100 000	1 000 000	5 000 000	10 000 000

Как оформить результат

```
Было:
GET /feed p95 = 1200ms

Сделал:
- cursor pagination
- composite index
- Redis cache
- denormalized feed_items

Стало:
GET /feed p95 = 80-150ms на локальной нагрузке
```

12. Репозиторий и структура проекта

Для портфолио удобнее monorepo: проще запускать docker-compose, проще держать общие библиотеки, проще показывать проект целиком.

```

devpulse/
├── services/
│   ├── auth/
│   ├── profile/
│   ├── posts/
│   ├── social_graph/
│   ├── feed/
│   ├── comments/
│   ├── reactions/
│   ├── notifications/
│   ├── analytics/
│   └── moderation/
├── gateway/
│   └── krakend.json
├── proto/
│   ├── profile.proto
│   ├── social_graph.proto
│   └── auth.proto
├── libs/
│   ├── common/
│   ├── observability/
│   └── messaging/
├── infra/
│   ├── docker-compose.yml
│   ├── prometheus/
│   ├── grafana/
│   ├── alertmanager/
│   ├── kafka/
│   ├── rabbitmq/
│   └── postgres/
├── load_tests/
│   ├── k6/
│   └── locust/
├── docs/
│   ├── architecture.md
│   ├── adr/
│   ├── api.md
│   ├── events.md
│   └── database.md
├── Makefile
└── README.md

```

Makefile минимум

```

make setup      # создать .venv и подготовить локальное окружение
make install    # поставить зависимости
make lint       # ruff / форматирование
make test       # pytest
make up         # docker compose up
make down       # docker compose down
make migrate    # alembic upgrade head по сервисам
make seed       # генерация тестовых данных
make load-test  # k6/locust сценарии

```

13. Очередность разработки

Этап	Что делаем	Результат
0. Фундамент	docker-compose, Makefile, ruff, pytest, env, healthcheck, Prometheus/Grafana пустым контуром	Проект запускается и проверяется единообразно
1. Auth + Profile	регистрация, логин, JWT, profile CRUD, PostgreSQL, Alembic, Redis, KrakenD routes	Можно зарегистрироваться, войти и получить профиль
2. Posts + Tags	Post Service, создание постов, теги, code snippets, outbox, Kafka producer	Посты создаются, PostCreated публикуется
3. Social Graph + Feed	follow/unfollow, Feed Service, Kafka consumer,	Появляется настоящая соцсетевая

	Redis cache, cursor pagination, fanout-on-write	механика
4. Reactions + Comments + Notifications	лайки, комментарии, RabbitMQ, Celery workers, retry, DLQ	Есть интерактивность и фоновые задачи
5. Observability	метрики, dashboards, alert rules, structured logging, correlation_id	Систему можно наблюдать как production-like окружение
6. High Load + PostgreSQL	seed data, load tests, индексы, EXPLAIN ANALYZE, cache strategy	Есть доказанная оптимизация
7. Документация	README, ADR, схемы, события, БД, trade-offs, скриншоты Grafana	Проект можно показывать работодателю

13.1 Этап 0 - фундамент

- docker-compose
- общий Makefile
- ruff
- pytest
- pre-commit
- общий формат env
- healthcheck каждого сервиса
- Prometheus + Grafana пустым контуром

13.2 Этап 1 - Auth + Profile

- Auth Service
- Profile Service
- JWT
- PostgreSQL
- Alembic
- Redis
- KrakenD routes
- базовые тесты

13.3 Этап 2 - Posts + Tags

- Post Service
- создание постов
- редактирование
- теги
- code snippets
- outbox table
- Kafka producer

13.4 Этап 3 - Social Graph + Feed

- follow/unfollow
- Feed Service
- Kafka consumer
- Redis cache
- cursor pagination
- fanout-on-write

13.5 Этап 4 - Reactions + Comments + Notifications

- лайки
- комментарии
- уведомления
- RabbitMQ
- Celery workers
- retry
- dead-letter queue

13.6 Этап 5 - Observability

- Prometheus metrics
- Grafana dashboards
- Alertmanager

- structured logging
- request_id / correlation_id

13.7 Этап 6 - High Load и PostgreSQL

- seed data
- load tests
- EXPLAIN ANALYZE
- индексы
- оптимизация feed
- оптимизация posts search
- Redis cache strategy

13.8 Этап 7 - Документация

- README
- архитектурная схема
- ADR
- описание Kafka events
- описание БД
- описание trade-offs
- скриншоты Grafana
- результаты нагрузочного теста

14. MVP, полная версия и то, что не тащить в первую итерацию

Минимальный MVP

- KrakenD;
- Auth Service;
- Profile Service;
- Post Service;
- Social Graph Service;
- Feed Service;
- Notification Service;
- PostgreSQL;
- Redis;
- Kafka;
- RabbitMQ;
- Celery;
- Prometheus;
- Grafana.

Полная версия после MVP

- Comment Service;
- Reaction Service;
- Analytics Service;
- Moderation Service;
- расширенные dashboards;
- нагрузочные отчёты;
- полная документация событий и БД.

Не тащить в первую версию

- Kubernetes;
- Service Mesh;
- OpenTelemetry traces;
- ClickHouse;
- Elasticsearch;
- S3/MinIO;
- GraphQL;

- сложный frontend;
- реальные email/SMS;
- ML-рекомендации;
- OAuth через GitHub.

Почему не тащить всё сразу

Иначе проект превратится в набор недоделанных интеграций. Ценность для портфолио появляется, когда есть завершённый end-to-end сценарий, метрики, тесты, документация и объяснимые trade-offs.

15. Чеклисты, риски и критерии готовности

Чеклист “не отклоняться”

- Каждый сервис имеет README: роль, API, БД, события, команды запуска.
- Каждый сервис имеет healthcheck.
- Каждый сервис отдаёт /metrics.
- У каждого сервиса своя БД или своя схема, без прямого доступа к чужим таблицам.
- Kafka используется для доменных событий, а не для фоновых задач.
- RabbitMQ/Celery используется для конкретных background jobs.
- Redis не хранит критичные данные как единственный источник правды.
- PostgreSQL-запросы с высокой нагрузкой проверяются через EXPLAIN ANALYZE.
- Все heavy-read endpoint'ы используют pagination.
- Feed не делает N+1 сетевых вызовов.
- Outbox используется там, где событие должно быть связано с записью в БД.
- README содержит сценарий запуска одной командой.
- Есть seed data и хотя бы один нагрузочный сценарий.
- Есть 2-3 скриншота Grafana и краткое описание метрик.

Риски

Риск	Как предотвратить
Слишком много сервисов сразу	Начать с MVP: Gateway, Auth, Profile, Post, Social Graph, Feed, Notifications
Distributed monolith	События через Kafka, batch gRPC, денормализация read models
Kafka/RabbitMQ используются хаотично	Развести event streaming и task queue
Нет доказанной пользы PostgreSQL-оптимизации	Добавить EXPLAIN ANALYZE до/после
Нет завершённого сценария	Сначала добиться register -> post -> feed -> notification
Документация в конце забыта	После каждого этапа обновлять docs/ и README

Критерии готовности MVP

- docker compose up поднимает gateway, сервисы, PostgreSQL, Redis, Kafka, RabbitMQ, Prometheus, Grafana;
- можно зарегистрировать пользователя;
- можно создать профиль;
- можно создать пост;
- можно подписаться на пользователя;
- после создания поста он появляется в ленте подписчика через Kafka-событие;
- уведомление создаётся через Kafka consumer и task отправляется через Celery/RabbitMQ;
- есть минимум 20-30 тестов на use cases и API;
- есть метрики и Grafana dashboard;
- есть README с архитектурной схемой и командами запуска.

Формулировка для портфолио

Разработал production-like микросервисную backend-систему DevPulse - социальную сеть для разработчиков. Система построена на FastAPI, PostgreSQL, Redis, Kafka, RabbitMQ, Celery и KrakenD API Gateway. Реализованы Auth, Profile, Posts, Social Graph, Feed и Notification services. Межсервисное взаимодействие частично построено на gRPC, доменные события передаются через

Kafka, фоновые задачи выполняются Celery worker'ами через RabbitMQ. Для ленты реализованы cursor pagination, Redis cache, fanout-on-write и оптимизация запросов PostgreSQL через индексы и EXPLAIN ANALYZE. Добавлены Prometheus metrics, Grafana dashboards и alert rules.

16. Первый рабочий спринт

Чтобы проект не расплылся, первая практическая цель - не "сразу вся соцсеть", а минимальный end-to-end контур. После него уже можно добавлять остальные сервисы без потери фокуса.

Цель спринта 1

- поднять monorepo и docker-compose;
- сделать Auth Service с регистрацией и логином;
- сделать Profile Service с базовым профилем;
- подключить PostgreSQL, Alembic, Redis;
- прокинуть маршруты через KrakenD;
- добавить healthcheck и /metrics;
- написать первые API/use-case тесты;
- оформить README с командами запуска.

Definition of Done спринта 1

- make up поднимает окружение без ручных действий;
- POST /auth/register создаёт пользователя;
- POST /auth/login выдаёт access/refresh token;
- GET /me возвращает текущего пользователя;
- PUT /profile обновляет профиль;
- GET /users/{id} возвращает публичный профиль;
- pytest проходит локально;
- Prometheus видит хотя бы Auth и Profile metrics;
- в README есть схема и пошаговый запуск.

Следующий шаг после PDF

Начинать стоит с репозитория, Makefile, docker-compose и двух сервисов: Auth + Profile. Только после этого добавлять Post Service и Kafka outbox.